A PROJECT REPORT ON

CAMPUS LOST & FOUND MANAGEMENT SYSTEM

Submitted to

MANIPAL UNIVERSITY JAIPUR

Towards the partial fulfillment for the Award of the Degree of

B. Tech Computer Science and Engineering

Session: 2026 – 2027 (Semester – III)

Course: Relational Database Management Systems Lab (CSC2131)

SUBMITTED BY

[Candidate Name 1] • Reg. No: [220905xxx]

[Candidate Name 2] • Reg. No: [220905xxx]

Under the guidance of

Dr. Satyabrata Roy / Dr. Lav Upadhyay

Course Coordinators, Department of Computer Science and Engineering

Manipal University Jaipur

Jaipur, Rajasthan (India)

ABSTRACT

The **Campus Lost and Found Management System** is a centralized, relational database-driven web application designed to digitize and manage the recovery lifecycle of misplaced belongings across Manipal University Jaipur. Built on **MySQL 8.0** and interfaced via a lightweight **Python Flask REST API** and modern desktop-style dashboard, the system eliminates informal, unstructured lost-and-found practices.

The relational schema is decomposed into **Third Normal Form (3NF)** across five core entities: `users`, `categories`, `items`, `claims`, and `activity_logs`. Key procedural features aligned with the **CSC2131 RDBMS Lab Curriculum** include an automated database **Trigger** (`trg_resolve_item_on_claim`) for automated item state transitions and audit logging upon claim approval, a **Stored Procedure** (`sp_approve_claim`) with ACID transaction control (`START TRANSACTION`, `COMMIT`), specialized **Database Views** (`vw_active_items`), and multi-table **SQL Joins** (including Self-Joins for spatial candidate matching). The platform ensures data integrity, protects claimant verification privacy, and guarantees institutional accountability.

Keywords: RDBMS, MySQL 8.0, 3NF Normalization, Database Triggers, Stored Procedures, Entity-Relationship Diagram, Relational Schema, Python Flask API.

TABLE OF CONTENTS

Section	Title	Page
1.	Introduction, Problem Statement & Objectives	3
2.	Scope of the Work & Product Scenarios	3
3.	Requirement Analysis & Environment Specifications	4
4.	System Design & Architecture	5
4.1	Entity-Relationship (ER) Conceptual Model	5
4.2	Relational Schema Specification	6
4.3	Database Normalization Analysis (UNF to 3NF)	7
4.4	Data Dictionary & Table Specifications	7
4.5	Advanced Objects: Triggers, Views & Stored Procedures	8
5.	Implementation & SQL Query Suite (Joins, Self-Join, Aggregations)	9
6.	Conclusion & References	10

1. INTRODUCTION

1.1 Background & Problem Statement

In academic institutions with multi-storey academic blocks, laboratories, libraries, and food courts, valuable personal belongings (laptops, student IDs, calculators, wallets, and keys) are frequently misplaced. Traditional recovery relies on chaotic social media groups or physical security registers, resulting in poor item recovery rates, identity fraud, and lack of accountability.

This project addresses these challenges through a centralized Relational Database Management System that systematically catalogs lost and found reports, conceals verification proof from public view, enforces relational integrity, and automates claim approvals via triggers.

1.2 Project Objectives

- **Relational Data Modeling:** Design a normalized Entity-Relationship (ER) model and Relational Schema adhering strictly to **Third Normal Form (3NF)**.
- **Database Implementation:** Implement the schema in **MySQL 8.0** using DDL scripts with Primary Keys, Foreign Keys, and integrity constraints.
- **Procedural SQL Programming:** Implement an `AFTER UPDATE` **Trigger** for automated item resolution, a **Stored Procedure** with transaction management, and **Views**.
- **Full-Stack Web Interface:** Connect MySQL to a Python Flask REST API and responsive desktop-style control panel.

2. SCOPE OF THE WORK & SCENARIOS

2.1 Scope of the Work

- **In-Scope:** User management (Students, Faculty, Staff, Admins), item cataloging (LOST/FOUND), spatial and category indexing, ownership claim submission, admin verification, automated trigger execution, and audit trail logging.
- **Out-of-Scope:** Off-campus recovery, physical locker hardware telemetry, and automated biometric dispensing.

2.2 Product Scenarios

- **Scenario 1 (Lost Report):** A student reports a lost Dell laptop in Lab 304, recording identifying stickers.
- **Scenario 2 (Found Report):** Campus security logs an ID card found in the Central Library with pickup desk details.
- **Scenario 3 (Claim Submission):** The authentic owner inspects the listing and submits ownership proof via web form.
- **Scenario 4 (Adjudication & Trigger):** Admin reviews proof and approves the claim. The MySQL trigger `trg_resolve_item_on_claim` automatically sets the item to `RESOLVED` and writes an audit log entry.

3. REQUIREMENT ANALYSIS

3.1 Functional Requirements

ID	Requirement	Expected Behavior
FR-01	User Registry	Maintain student, staff, and admin accounts with unique emails and contact details.
FR-02	Categorization	Standardized classification into Electronics, IDs, Wallets, and Keys.
FR-03	Item Reporting	Register LOST and FOUND items with incident dates, locations, and descriptions.
FR-04	Search & Filter	Real-time querying by report type (LOST/FOUND), category, and keyword.
FR-05	Ownership Claims	Allow students to submit ownership proof for found items.
FR-06	Admin Adjudication	Authorized administrators review proofs and record approval/rejection decisions.
FR-07	Automated Trigger	Automatically updates item status to `RESOLVED` upon claim approval.
FR-08	Audit Logging	System actions recorded into `activity_logs` for institutional traceability.

3.2 Non-Functional & Environmental Specifications

- **Data Integrity:** Enforce referential integrity (`ON UPDATE CASCADE`, `ON DELETE RESTRICT/CASCADE`).
- **ACID Transactions:** Isolated claim processing preventing concurrent race conditions.
- **Technology Stack:** MySQL 8.0 (InnoDB), Python 3 Flask, HTML5/CSS3/JavaScript Fetch API.

3.3 Use Case Scenarios

Actor	Action / Use Case	Precondition / Result
Student	Post Lost Item	Incident details stored with `status = 'ACTIVE'`.
Student / Staff	Post Found Item	Found item registered with safe storage location recorded.
Claimant	Submit Claim	Claim record created with `status = 'PENDING'`; item set to `CLAIM_PENDING`.
Administrator	Approve Claim	Trigger fires: claim set to `APPROVED`, item set to `RESOLVED`, audit logged.

4. SYSTEM DESIGN

4.1 Three-Tier Architecture

The system uses a **Three-Tier Architecture**: (1) **Presentation Tier:** Desktop-style responsive dashboard; (2) **Application Tier:** Python Flask REST API executing direct parameterized SQL queries; (3) **Database Tier:** MySQL 8.0 relational database with ACID-compliant InnoDB tables, views, and triggers.

4.2 Entity-Relationship (ER) Conceptual Model

The conceptual model consists of five strong entities connected through primary and foreign key constraints:

- **USERS (1) — reports — (N) ITEMS:** A user can report multiple lost/found items.
- **CATEGORIES (1) — classifies — (N) ITEMS:** Each item belongs to exactly one category.
- **ITEMS (1) — has — (N) CLAIMS:** A found item can receive multiple ownership claims.
- **SYSTEM (1) — logs — (N) ACTIVITY_LOGS:** Automated triggers insert audit logs.

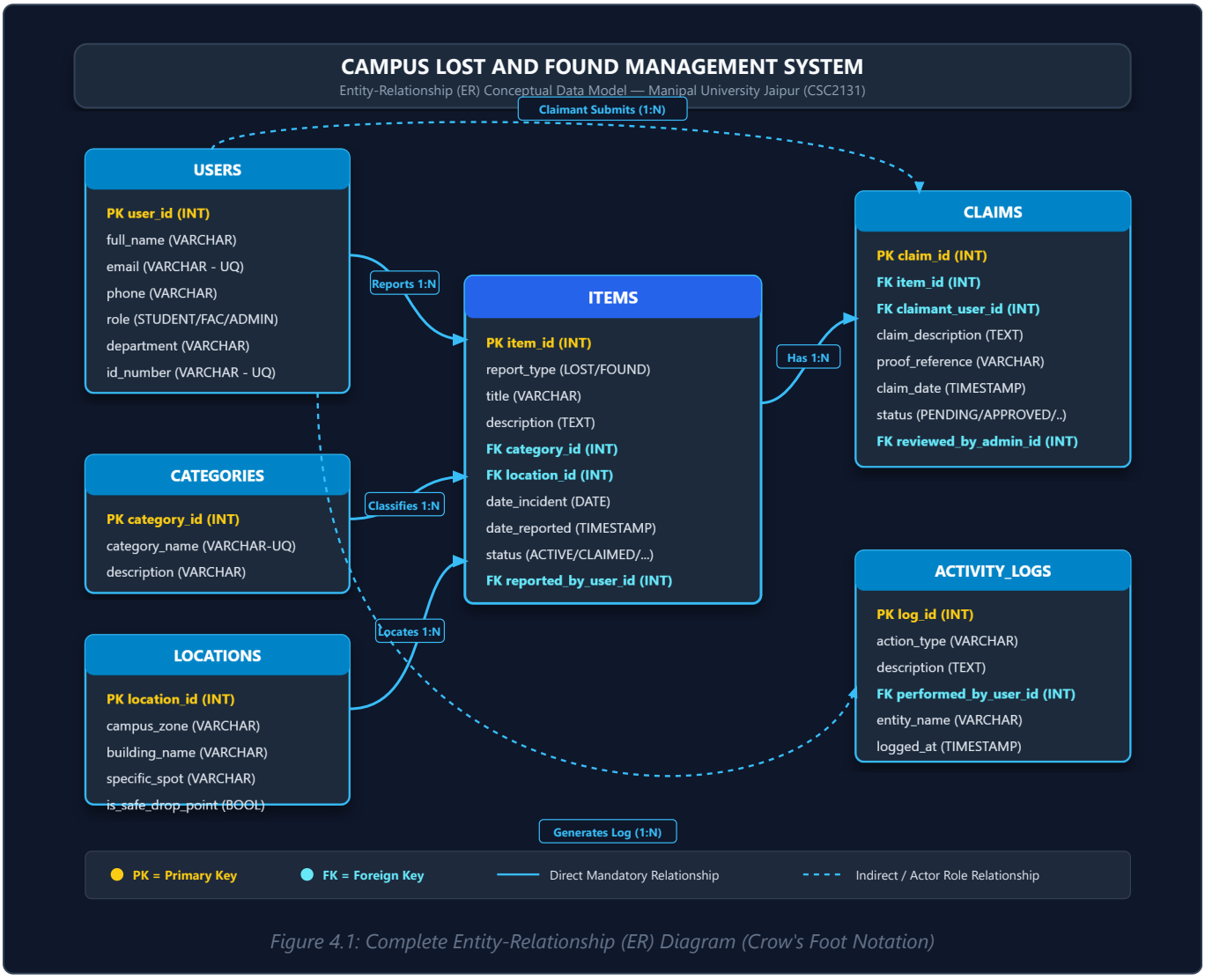


Figure 4.1: Complete Entity-Relationship (ER) Diagram (Crow's Foot Notation)

4.3 Relational Schema Mapping

The logical schema maps entities into relational tables. Primary Keys are underlined and Foreign Keys are marked:

- **USERS** (user_id, full_name, email, phone, role)
- **CATEGORIES** (category_id, category_name)
- **ITEMS** (item_id, report_type, title, description, category_id^{FK}, location, date_incident, status, reported_by^{FK})
- **CLAIMS** (claim_id, item_id^{FK}, claimant_name, claimant_contact, proof_details, status)
- **ACTIVITY_LOGS** (log_id, action_type, details, log_time)

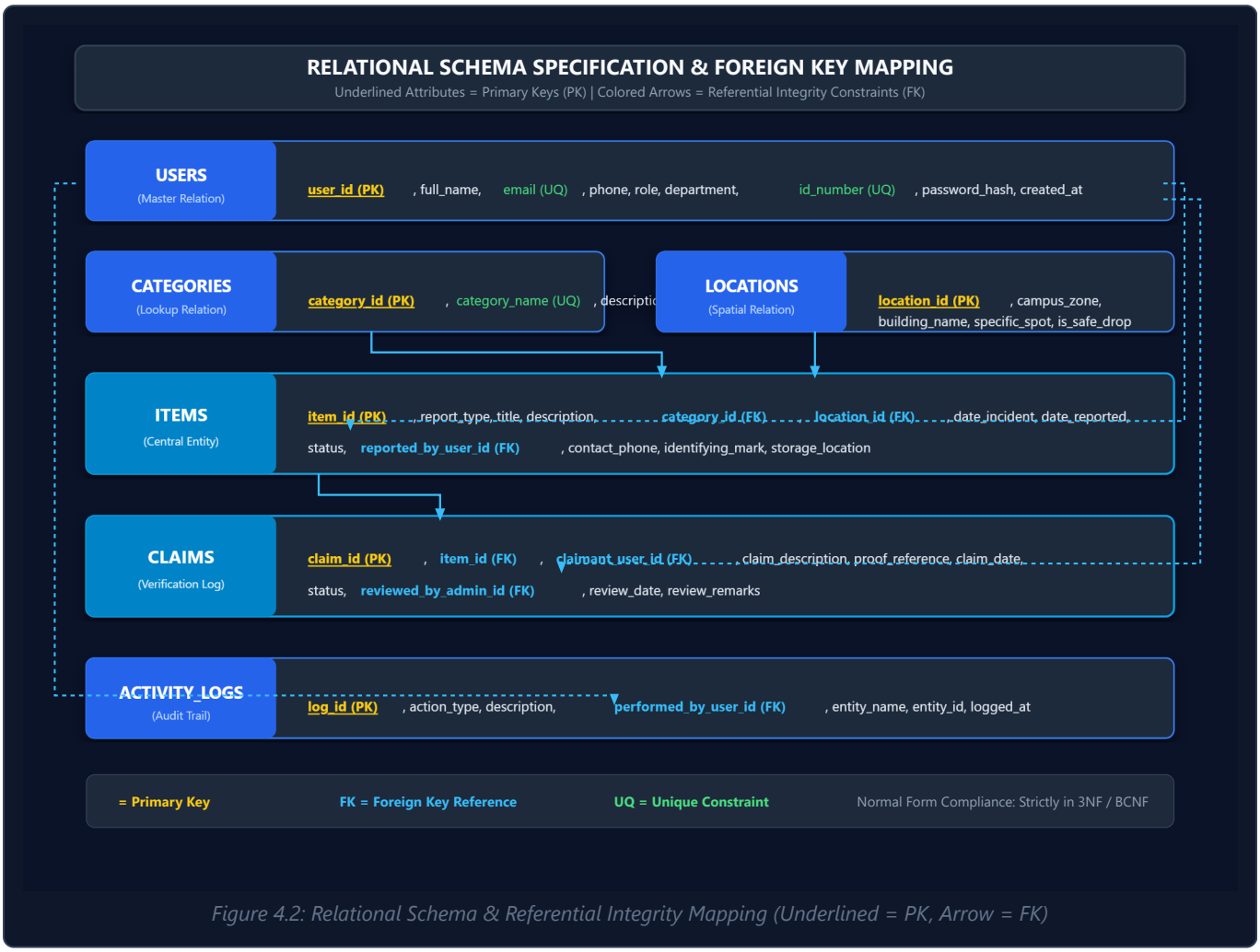


Figure 4.2: Relational Schema & Referential Integrity Mapping (Underlined = PK, Arrow = FK)

4.4 Database Normalization Analysis (UNF to 3NF)

1. Unnormalized Form (UNF): A flat table combining all attributes has repeating claim groups and duplicate category names:
`UNF = { ItemID, Title, CatName, Location, Date, ReporterName, ReporterPhone, { ClaimantName, Proof, ClaimStatus }* }`

2. First Normal Form (1NF): Eliminate repeating groups by separating Claims into an independent table with atomic attribute domains:
`ITEMS_1NF = { item_id, report_type, title, description, category_name, location, date_incident, status, reporter_id, reporter_name, phone }`
`CLAIMS_1NF = { claim_id, item_id, claimant_name, claimant_contact, proof_details, status }`

3. Second Normal Form (2NF): All tables have single-attribute primary keys (`item_id`, `claim_id`), eliminating all partial functional dependencies.

4. Third Normal Form (3NF): Remove transitive dependencies where non-key attributes depend on other non-key attributes:
`item_id → category_id → category_name` and `item_id → reported_by → full_name, phone`
Extracted into independent lookup relations: `CATEGORIES` and `USERS`.
Conclusion: All relations strictly satisfy **Third Normal Form (3NF) and BCNF**.

4.5 Data Dictionary

Table	Column	Type	Constraint	Description
users	user_id	INT	PK, AUTO_INC	Unique user ID
users	full_name	VARCHAR(100)	NOT NULL	User full name
users	email	VARCHAR(100)	NOT NULL, UQ	University email
categories	category_id	INT	PK, AUTO_INC	Category ID
categories	category_name	VARCHAR(50)	NOT NULL, UQ	Category title (Electronics, IDs)
items	item_id	INT	PK, AUTO_INC	Unique item ID
items	report_type	ENUM	NOT NULL	'LOST' or 'FOUND'
items	category_id	INT	FK	Ref categories(category_id)
items	status	ENUM	DEFAULT 'ACTIVE'	'ACTIVE', 'CLAIM_PENDING', 'RESOLVED'
items	reported_by	INT	FK	Ref users(user_id)
claims	claim_id	INT	PK, AUTO_INC	Claim ID
claims	item_id	INT	FK	Ref items(item_id)
claims	status	ENUM	DEFAULT 'PENDING'	'PENDING', 'APPROVED', 'REJECTED'
activity_logs	log_id	INT	PK, AUTO_INC	Audit Log ID

4.6 Advanced Database Objects

1. Database Trigger (Lab 12 / CO CSC2131.5)

Automatically updates item status to RESOLVED and creates an audit log entry upon claim approval:

```
DELIMITER $$
CREATE TRIGGER trg_resolve_item_on_claim
AFTER UPDATE ON claims
FOR EACH ROW
BEGIN
    IF NEW.status = 'APPROVED' AND OLD.status != 'APPROVED' THEN
        -- 1. Update item status to RESOLVED
        UPDATE items SET status = 'RESOLVED' WHERE item_id = NEW.item_id;

        -- 2. Log audit trail
        INSERT INTO activity_logs (action_type, details)
        VALUES ('CLAIM_APPROVED', CONCAT('Claim #', NEW.claim_id, ' approved for Item #', NEW.item_id));
    END IF;
END$$
DELIMITER ;
```

2. Database View (Lab 10 / CO CSC2131.3)

Multi-table joined view providing active item listings for the student portal:

```
CREATE VIEW vw_active_items AS
SELECT i.item_id, i.report_type, i.title, c.category_name, i.location,
       i.date_incident, i.status, u.full_name AS reported_by_name
FROM items i
JOIN categories c ON i.category_id = c.category_id
JOIN users u ON i.reported_by = u.user_id
WHERE i.status != 'RESOLVED';
```

3. Stored Procedure with Transaction Control (Lab 12 / CO CSC2131.4)

Executes atomic claim approval within an isolated transaction (START TRANSACTION, COMMIT):

```
DELIMITER $$
CREATE PROCEDURE sp_approve_claim(IN p_claim_id INT)
BEGIN
    START TRANSACTION;
        UPDATE claims SET status = 'APPROVED' WHERE claim_id = p_claim_id;
    COMMIT;
END$$
DELIMITER ;
```

5. IMPLEMENTATION & SQL QUERY DEMONSTRATIONS

5.1 Course Outcome (CO) Alignment Matrix

CO Code	Syllabus Outcome	Project Implementation Deliverable
CSC2131.1	ER Modeling & Normalization	ER Diagram, Relational Schema, and mathematical 3NF Proof.
CSC2131.2	Schema Creation using DDL	5 Tables with Primary, Foreign Key, and Unique constraints.
CSC2131.3	DML, Joins & Views	Multi-table Inner Joins, Self-Joins, and View `vw_active_items`.
CSC2131.4	Transaction Control (TCL)	`START TRANSACTION` and `COMMIT` in `sp_approve_claim`.
CSC2131.5	Procedural Logic & Triggers	Trigger `trg_resolve_item_on_claim` automating state updates.

5.2 Key SQL Query Demonstrations

Query 1: Multi-Table Inner Join (Lab 8)

```
SELECT i.item_id, i.report_type, i.title, c.category_name, i.location, u.full_name AS reported_by
FROM items i
JOIN categories c ON i.category_id = c.category_id
JOIN users u ON i.reported_by = u.user_id
WHERE i.status = 'ACTIVE';
```

Query 2: SQL Self-Join for Spatial Candidate Matching (Lab 8)

```
SELECT lost.item_id AS lost_id, lost.title AS lost_item,
       found.item_id AS found_id, found.title AS found_item, lost.location
FROM items lost
JOIN items found ON lost.location = found.location AND lost.category_id = found.category_id
WHERE lost.report_type = 'LOST' AND found.report_type = 'FOUND' AND lost.status = 'ACTIVE';
```

Query 3: Conditional Metric Aggregation (Lab 7)

```
SELECT COUNT(*) AS total_items,
       SUM(CASE WHEN report_type = 'LOST' THEN 1 ELSE 0 END) AS total_lost,
       SUM(CASE WHEN report_type = 'FOUND' THEN 1 ELSE 0 END) AS total_found,
       SUM(CASE WHEN status = 'RESOLVED' THEN 1 ELSE 0 END) AS total_resolved
FROM items;
```

6. CONCLUSION & REFERENCES

6.1 Conclusion

The **Campus Lost and Found Management System** delivers an efficient, robust relational database solution for Manipal University Jaipur. Decomposed into **3NF**, the schema prevents insertion, update, and deletion anomalies. By implementing database triggers, views, stored procedures, and complex SQL joins, the project demonstrates how fundamental RDBMS principles solve real-world campus operational challenges.

6.2 Future Scope

- **Computer Vision Matching:** Implement image similarity embedding to cross-match photos of lost and found items.
- **Automated Notification Gateway:** Trigger SMS/Email notifications to students when candidate matches are logged.
- **IoT Smart Locker Integration:** Enable automated locker-based handovers for verified claims.

6.3 References

1. **Silberschatz, A., Korth, H. F., & Sudarshan, S.** (2021). *Database System Concepts* (7th ed.). McGraw Hill Education.
2. **Elmasri, R., & Navathe, S. B.** (2017). *Fundamentals of Database Systems* (7th ed.). Pearson Education.
3. **Beaulieu, A.** (2020). *Learning SQL: Generate, Manipulate, and Retrieve Data* (3rd ed.). O'Reilly Media.
4. **Oracle Corporation** (2026). *MySQL 8.0 Reference Manual: Triggers, Views, and Stored Programs*.
5. **Manipal University Jaipur** (2026). *Course Hand-out: Relational Database Management Systems Lab (CSC2131)*. Department of Computer Science and Engineering.